Project Proposal

A Hallucination-Resistant Retrieval-Augmented Generation System for Grounded Financial Query Answering

Template version: V0, 03/25/2026
Course: LLMs: A Hands-on Approach, CCE IISc
Team Members: Gangadhar Singh Shiva, shivagangadharsingh@gmail.com]
Date: Mar,29,2026

1. Problem Statement

Problem: Large Language Models (LLMs) hallucinate factually incorrect financial information at rates that make raw LLM output unsuitable for production financial applications, yet no standardised, open-source pipeline exists that combines real-time retrieval, multi-layer hallucination prevention, and agentic post-processing in a single deployable system.

Relevance: In financial applications, a single fabricated stock figure or erroneous SEC filing reference can constitute material misinformation with legal and regulatory consequences. Existing commercial systems (Bloomberg Terminal, Refinitiv) use proprietary, closed architectures. Open-source financial LLM projects such as FinGPT and BloombergGPT demonstrate strong domain adaptation but lack retrieval-augmented grounding and explicit hallucination prevention mechanisms. The gap between LLM capability and production-grade reliability in finance is an open, high-impact engineering and research problem.

Domain: RAG (Retrieval-Augmented Generation), Inference Optimization, and Agents — specifically the design of multi-node agentic pipelines with real-time external data retrieval, confidence-gated reflection, and layered hallucination prevention for knowledge-intensive financial query answering.

2. Proposed Approach

Core idea: To design and implement FinRAG, a production-grade RAG system that routes financial queries through a seven-layer defence pipeline: **intent classification**, **entity validation**, **data quality filtering,** v**ector similarity gating**, **ticker-aware retrieval**, **FinBERT-based confidence scoring, and self-reflective answer validation.** The system fetches live financial data from three real-time sources (Alpha Vantage market data, SEC EDGAR filings, and social sentiment) via **Model Context Protocol (MCP) servers**, embeds and stores retrieved documents in **ChromaDB**, and assembles **grounded answers** by direct document concatenation — eliminating LLM generation from the core answer step to remove the primary hallucination source. Three post-retrieval agents (FinBERT FinAnalystAgent, FinReflectionAgent confidence gate, FinScribeAgent report formatter) enrich responses with domain-specific analysis.

Project type: Working demo — a fully deployed, containerised system on Kubernetes (Minikube) with a REST API, integration test suite, and benchmark comparison tool.

Models:

- ProsusAI/finbert (FinBERT) — financial sentiment classification (positive/negative/neutral) with confidence scoring
- sentence-transformers/all-MiniLM-L6-v2 — 384-dimensional sentence embeddings for document indexing and cosine similarity retrieval
- GPT-4o / Llama-3 (baseline comparison) — plain LLM pipeline used as the hallucination baseline in benchmark evaluation

Datasets / Benchmarks:

- Alpha Vantage TIME_SERIES_DAILY API — real-time OHLCV stock price data for AAPL, MSFT, TSLA, NVDA
- SEC EDGAR public REST API — 10-K and 10-Q filing metadata (no authentication required)
- Twitter/X API v2 with VADER sentiment scoring — social sentiment signals (25 req/day free tier)
- Custom adversarial test suite — 17 queries across 6 categories (factual, advisory, non-existent entity, fabricated year, confidential, hallucination probe) with labelled expected outcomes

What is novel: FinRAG introduces a ticker-guard post-retrieval filter that addresses cross-ticker vector store contamination — a previously undocumented hallucination pathway in ChromaDB-backed RAG systems — and demonstrates through targeted adversarial probe queries that standard hallucination metrics (sentence overlap) systematically fail to detect this class of grounding failure, necessitating purpose-designed probe-based evaluation alongside conventional metrics.

3. Related Work

#	Reference (title, authors, year)	How your project differs from it
1	Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.	FinRAG extends the base RAG paradigm with seven sequential hallucination guard layers, ticker-aware post-retrieval filtering, and a self-reflective confidence gate — none of which appear in the original RAG formulation.
2	Yang et al. (2020). FinBERT: A Pretrained Language Model for	FinRAG uses FinBERT as one component (AnalystAgent sentiment classification) within a broader

	Financial Communications. arXiv:2006.08097.	multi-node pipeline, rather than as a standalone classifier. We also integrate FinBERT confidence as a gate that controls report generation.
3	Wu et al. (2023). BloombergGPT: A Large Language Model for Finance. arXiv:2303.17564.	BloombergGPT is a domain-adapted LLM with no retrieval or hallucination prevention layer. FinRAG is architecture-level complementary: it could use BloombergGPT as the backbone LLM while adding the retrieval and guard infrastructure BloombergGPT lacks.
4	Asai et al. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv:2310.11511.	Self-RAG implements self-critique via additional fine-tuned tokens. FinRAG implements the self-critique paradigm without fine-tuning, using FinBERT confidence and cosine similarity as explicit, interpretable gate signals rather than learned reflection tokens.
5	Yao et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.	ReAct interleaves reasoning and tool use in a single LLM call. FinRAG separates retrieval, validation, and reasoning into discrete, independently testable pipeline nodes using LangGraph DAG orchestration, enabling node-level ablation and targeted patching.
6	Ji et al. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys.	Ji et al. taxonomise hallucination types. FinRAG directly implements defences against all three financial hallucination categories identified in Ji et al.: factual errors (similarity gate), temporal errors (year whitelist guard), and attribution errors (ticker guard + reflection agent).

4. Expected Deliverables

Code repository:

- FastAPI application with complete LangGraph pipeline (11 nodes) — main.py
- Three MCP data server implementations (market, SEC, social) with Docker images
- Three agentic modules: AnalystAgent (FinBERT), ReflectionAgent (confidence gate), ScribeAgent (report generation)
- Kubernetes deployment manifests, ConfigMaps, Secrets, and automated deploy.sh script
- Benchmark tool for BASELINE vs OPTIMIZED pipeline comparison on hallucination rate, faithfulness, grounded rate, and latency
- ChromaDB vector store with persistent embedding layer and deduplication

Report: Full technical report covering system architecture, pipeline node design, agentic integration, Kubernetes deployment configuration, hallucination vulnerability discovery and patching, benchmark methodology and results, and evaluation analysis.

Includes dataset EDA, variable catalogue, data quality issue tracking, and identified remaining risks.

Demo: Live demo via Kubernetes NodePort service (localhost:8080) accessible through kubectl port-forward; recorded video walkthrough demonstrating factual queries, adversarial probe blocking, and real-time hallucination prevention across all seven guard layers.

5. Evaluation Plan

Accuracy — Hallucination rate and faithfulness score:

- Hallucination rate: fraction of answer sentences not semantically supported by any retrieved document (cosine similarity < 0.65 against retrieved docs). Target: 0.0 on factual queries.
- Faithfulness score: 1 − hallucination_rate. Target: 1.0 on grounded responses.
- Grounded rate: fraction of factual queries producing a non-blocked answer. Target: ≥ 85%.
- Blocked rate: fraction of adversarial/out-of-scope queries correctly blocked. Target: ≥ 75%.

Hallucination probe evaluation:

- Three adversarial probe queries designed to bypass standard guards: (1) stale vector store probe (historical date query), (2) social sentinel bypass probe (placeholder data injection), (3) cross-ticker contamination probe (multi-company sequence attack). Target: 0 probes leaking post-fix.

Latency — Inference time:

- End-to-end wall-clock latency from POST /analyze to JSON response. Measured per query category. OPTIMIZED pipeline target: < 10s average on factual queries.
- BASELINE vs OPTIMIZED latency comparison. Expected: OPTIMIZED faster due to vector retrieval vs full LLM generation.

Resource usage — Memory and compute:

- Kubernetes pod memory limits: 2Gi for API pod. Monitored via kubectl top pods.
- FinBERT and sentence-transformer model loading time tracked; singleton pattern validated for per-request overhead elimination.

Ablation studies — Component contribution analysis:

- Guard layer removal study: systematically disable each of the seven guard nodes and measure change in blocked rate and hallucination probe leak count.

- Similarity threshold ablation: test thresholds {0.40, 0.55, 0.65, 0.75} and measure recall-precision trade-off on grounded rate vs hallucination rate.
- Confidence gate ablation: test FinBERT confidence thresholds {0.50, 0.60, 0.70, 0.80} and measure report generation rate vs low-confidence report suppression accuracy.
- Ticker guard impact: compare cross-ticker contamination probe results pre- and post-ticker_guard_node to quantify the vulnerability severity and fix effectiveness.

6. Timeline

Week	Plan
1	Literature review (RAG, FinBERT, LangGraph, MCP, hallucination prevention). Environment setup: Minikube, Docker, Alpha Vantage API key. Initial FastAPI scaffolding and MCP server stubs.
2	Core pipeline implementation: intent_node, fetch_node, validate_node, store_node, retrieve_node, evaluate_node, answer_node. ChromaDB integration with all-MiniLM-L6-v2 embeddings.
3	Agentic integration: AnalystAgent (FinBERT), ReflectionAgent (confidence gate), ScribeAgent. LangGraph conditional edge for confidence routing. Kubernetes deployment with deploy.sh.
4	Integration test suite development (17 queries, 6 categories). Hallucination probe design and execution. Vulnerability identification and patching (social sentinel, ticker contamination). Benchmark tool implementation.
5	Ablation studies: threshold sweeps, guard layer removal experiments. Benchmark analysis (BASELINE vs OPTIMIZED). Report writing, demo recording, repository finalisation and documentation.